

Fluent Interface Design Pattern in C# with Examples

In this article, I will discuss the **Fluent Interface Design Pattern in C#** with Examples. Please read our previous article discussing the [Builder Design Pattern in C#](#) with Examples. The Fluent Interface Design Pattern falls under the category of the Creational Design Pattern. As part of this article, we will discuss the following pointers.

1. [What is the Fluent Interface Design Pattern?](#)
2. [Understanding Method Chaining in C#.](#)
3. [Implementing Fluent Interface Design Pattern in C#.](#)
4. [When to use the Fluent Interface Design Pattern?](#)

Note: Fluent Interfaces and Method Chaining are related. One is a concept, and the other is its implementation.

What is the Fluent Interface Design Pattern?

The Fluent Interface Design Pattern in C# is a method for designing object-oriented APIs that provides more readable and easily maintainable code. It is based on the concepts of method chaining, where each method returns an object itself, allowing multiple actions to be invoked in a single statement sequence. It is useful for creating an API where the code reads like a series of natural language statements, enhancing readability and ease of use.

Method Chaining: Each method returns an instance of the object itself, typically this, which allows for the chaining of multiple method calls in a single statement.

How to Implement Fluent Interface Design Pattern in C#?

Let us understand How to Implement the Fluent Interface Design Pattern in C# with an Example. Let's say we have the following Employee class.

```
public class Employee
{
    public string FullName { get; set; }
    public DateTime DateOfBirth { get; set; }
    public string Department { get; set; }
    public string Address { get; set; }
}
```

If we want to consume the above Employee class, we generally create an instance of the Employee class and set the respective properties as shown below.

```
Employee employee = new Employee();
employee.FullName = "Anurag Mohanty";
employee.DateOfBirth = Convert.ToDateTime("10/10/1992");
employee.Department = "IT";
employee.Address = "Mumbai-India";
```

The Fluent interfaces simplify our object consumption code by making it simpler, readable, and easy to use. Would it not be nice to set the object properties as shown below?

```
Employee employee = new Employee();

employee.NameOfTheEmployee("Anurag Mohanty")
    .Born("10/10/1992")
    .WorkingOn("IT")
    .StaysAt("Mumbai-India");
```

Creating such interfaces is like speaking a sentence that would make the class consumption code more simple and readable. Now, the next thing is how to achieve this. To achieve this, we have something called Method Chaining in C#.

What is Method Chaining in C#?

Method Chaining in C# is a common technique where each method returns an object, and all these methods can be chained together to form a single statement. To achieve this, first, we need to create a wrapper class around the Employee class, as shown below.

```
public class FluentEmployee
{
    private Employee employee = new Employee();

    public FluentEmployee NameOfTheEmployee(string FullName)
    {
        employee.FullName = FullName;
        return this;
    }

    public FluentEmployee Born(string DateOfBirth)
    {
        employee.DateOfBirth = Convert.ToDateTime(DateOfBirth);
        return this;
    }

    public FluentEmployee WorkingOn(string Department)
    {
        employee.Department = Department;
        return this;
    }

    public FluentEmployee StaysAt(string Address)
    {
        employee.Address = Address;
        return this;
    }
}
```

As you can see, we have created methods for each property. Also, notice that the method's return type is set to the FluentEmployee. Now, the client is going to consume the above fluent interface. So, with the above FluentEmployee class in place, the client code should look as shown below.

```
FluentEmployee obj = new FluentEmployee();

obj.NameOfTheEmployee("Anurag Mohanty")
  .Born("10/10/1992")
  .WorkingOn("IT")
  .StaysAt("Mumbai-India");
```

Example to Implement Fluent Interface Design Pattern using C#

Whatever we have discussed so far, let us implement it using the Fluent Interface Design Pattern in C#. First, we need to create a class file named **Employee.cs** and then copy and paste the following code into it. This is a simple class with few properties. We aim to set these property values using Method Chaining.

```
using System;
namespace FluentInterfaceDesignPattern
{
    public class Employee
    {
        public string FullName { get; set; }
        public DateTime DateOfBirth { get; set; }
        public string Department { get; set; }
        public string Address { get; set; }
    }
}
```

Creating Wrapper:

To set the above Employee class Property Values using Method Chaining, we must create a Wrapper class around the Employee class. So, create a class file named **FluentEmployee.cs** and copy and paste the following code. This is going to be our Wrapper class. If you notice, here, we have created an instance of the Employee class. For each Employee class property, here we have created a corresponding method, and using those methods, we are setting the Employee object property values. Further, if you notice, the return type of each method (except ShowDetails) is FluentEmployee type, which is important. Because of this return type (which is returning the FluentEmployee instance), we can call the FluentEmployee class methods (except ShowDetails) one after another using the dot operator. The ShowDetails method is not for setting the values; it is used to display them.

```
using System;
namespace FluentInterfaceDesignPattern
{
    public class FluentEmployee
    {
        private Employee employee = new Employee();
        public FluentEmployee NameOfTheEmployee(string FullName)
        {
            employee.FullName = FullName;
            return this;
        }
        public FluentEmployee Born(string DateOfBirth)
        {
            employee.DateOfBirth = Convert.ToDateTime(DateOfBirth);
            return this;
        }
    }
}
```

```

    public FluentEmployee WorkingOn(string Department)
    {
        employee.Department = Department;
        return this;
    }
    public FluentEmployee StaysAt(string Address)
    {
        employee.Address = Address;
        return this;
    }
    public void ShowDetails()
    {
        Console.WriteLine($"Name: {employee.FullName}, \nDateOfBirth:
{employee.DateOfBirth}, \nDepartment: {employee.Department}, \nAddress:
{employee.Address}");
    }
}
}

```

Using Wrapper in the Client Code:

In our example, the Program class is going to be the Client. So, from the Client, we need to create an instance of the FluentEmployee and call the FluentEmployee class methods one after another using the dot (.) operator, which is nothing but Method Chaining. So, modify the Main method of the Program class as follows.

```

using System;
namespace FluentInterfaceDesignPattern
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create an Instance of Wrapper class i.e. FluentEmployee
            FluentEmployee obj = new FluentEmployee();
            //Call Methods one by one using dot Operator whose Return Type
            is FluentEmployee
            obj.NameOfTheEmployee("Anurag Mohanty")
                .Born("10/10/1992")
                .WorkingOn("IT")
                .StaysAt("Mumbai-India");
            //To See the Details call the ShowDetails Method
            obj.ShowDetails();
            Console.Read();
        }
    }
}

```

Output:

```

Name: Anurag Mohanty,
DateOfBirth: 10-10-1992 00:00:00,
Department: IT,
Address: Mumbai-India

```

When to Use Fluent Interface Design Pattern in C#?

The Fluent Interface Design Pattern in C# is beneficial in the following scenarios:

- **Configure Complex Objects:** When an object requires multiple optional and mandatory configurations, a fluent interface simplifies the process, making it clear and straightforward.
- **Build SQL-like Queries:** Fluent interfaces are ideal for designing APIs for building complex queries or operations, similar to how LINQ works in C#. This allows for constructing queries in a way that is both expressive and easy to modify.
- **Create Nested Configuration Structures:** They are excellent for scenarios where you need to build nested configurations, as each level of nesting can return contexts that allow for further configurations.

In the next article, I will discuss [Real-Time Examples of the Fluent Interface Design Pattern in C#](#). In this article, I try to explain the **Fluent Interface Design Pattern in C#** with Examples. I hope you understand the need for and use of the Fluent Interface Design Pattern.